

Money Was the Move

Reading Coindroids in the Defcoin Chain

During DEF CON 25, Coindroids reported that it accounted for **71% of all transactions on the Defcoin network**. In the same event window, the Defcoin realm saw **3,233 attacks**, **126.664 DFC** in attack volume, **521 item purchases**, **81 defensive actions**, **202 droid deaths**, and **227.046 DFC** paid back out. Those are not vibes. They are a testable forensic target. Source: [Coindroids blog](#), “Coindroids at DEF CON 25”.

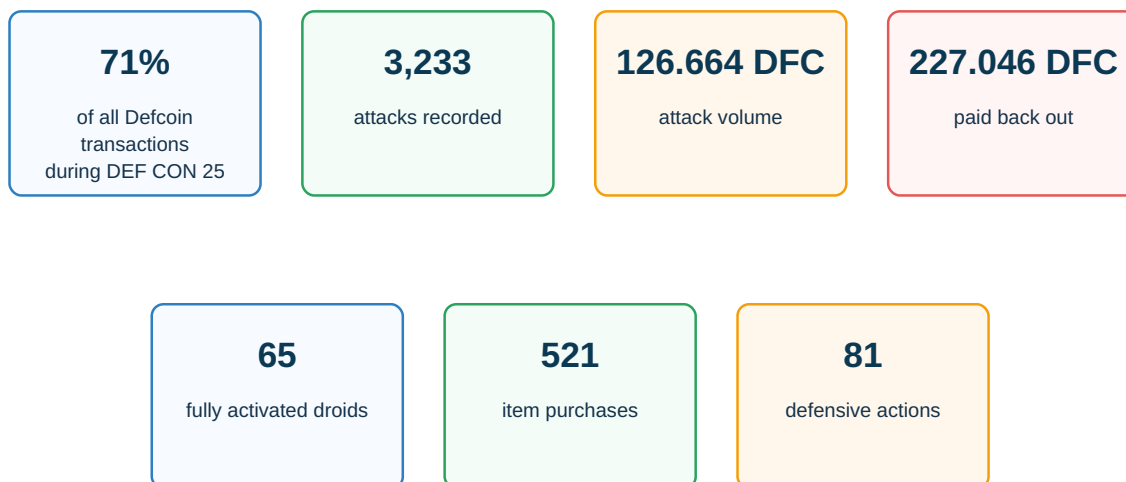
The problem is simple enough to state and hard enough to make interesting: can a Defcoin utility app look at ordinary UTXO transactions and reconstruct the game that produced them?

Not merely *flag* Coindroids. Reconstruct it.

Who was attacking. Which droids were targets. Which transactions bought time, defense, health, or position. Which payouts were ordinary settlements and which were jackpot-like prizes. Which activity looked manual and which looked automated. Which share of the chain was Coindroids, and what remained outside it.

The Defcoin chain becomes the witness. Coindroids provides the grammar. A forensics utility provides the translation.

Figure 1. The first clue: one game leaves most of the footprints



A forensic utility can start here: published event totals become benchmark tests for local chain classification.

Figure 1. All numbers in this figure are from the DEF CON 25 Coindroids recap. Source: [S2: Coindroids blog](#), “Coindroids at DEF CON 25”.

The Forensic Question

The first conflict is not technical. It is interpretive. A blockchain explorer can show that money moved. It cannot automatically tell whether that movement was an attack, a shield, a registration, an item purchase, a change output, an error recovery, or a payout. Coindroids makes that distinction meaningful because the game treated transactions as actions. The 2014 introduction states that attacks, item purchases, and upgrades were performed by sending currency to specific game-controlled addresses. Source: [Coindroids blog, “Introducing Coindroids \(2014\)”](#).

A good Defcoin utility therefore needs a stronger index than a normal explorer. It needs to classify outputs by game role, tie them to block height and droid identity, track delayed payouts, and expose its confidence reasons.

The report follows the investigation the utility itself should perform: begin with the published anomaly, identify the grammar of the game, separate the noisy 2014 evidence from the quieter 2017+ model, then turn those clues into implementation rules.

Figure 2. Evidence timeline: how the trail changes

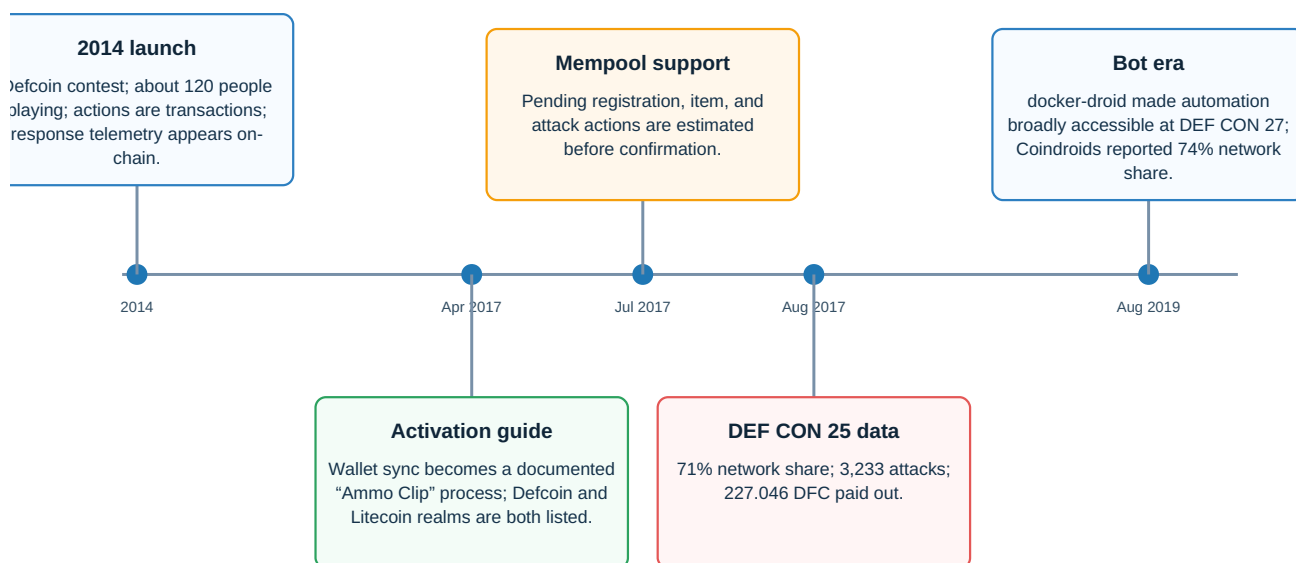


Figure 2. Timeline sources: 2014 launch and telemetry from S1: [Coindroids blog, “Introducing Coindroids \(2014\)”](#); wallet sync from S6: [Coindroids blog, “Creating a Droid”](#); mempool support from S3: [Coindroids blog, “Defcoin Mempool support added”](#); DEF CON 25 totals from S2: [Coindroids blog, “Coindroids at DEF CON 25”](#); DEF CON 27 automation and network share from S8: [Coindroids blog, “Coindroids @ Defcon 27”](#).

The Trail Begins: 2014, When the Chain Spoke Back

The early Coindroids system was unusually visible on-chain. The 2014 post describes wallet registration through a one-time-use address: for Defcoin, a user sent **0.01 DFC**, and the system returned it in the next block. It also describes transaction responses in which Coindroids tried to acknowledge actions with additional transactions. For attacks, those responses could include many tiny outputs whose destination addresses and output amounts encoded attacker level, attacker health, defender level, defender health, damage, and counter-damage. Source: [Coindroids blog, “Introducing Coindroids \(2014\)”](#).

This is a gift to a forensic tool. It means some early Coindroids transactions are not merely identifiable; they are readable. A utility can scan for known cosmetic response addresses, decode tiny amounts as game-state values, and display the old transaction as a combat report.

Figure 4. The 2014 clue: tiny outputs can become readable battle telemetry

Cosmetic address marker	Tiny output	Decoded meaning
D AtkLv...	0.00000020	Attacker level = 20
D AtkHP...	0.00000839	Attacker HP = 839
D DfndLv...	0.00000018	Defender level = 18
D DfndHP...	0.00000002	Defender HP = 2
DAMAGE...	0.00001287	Damage dealt = 1287
D CDAMG...	0.00001019	Counter-damage = 1019

Implementation rule: flag transactions with several known Coindroids cosmetic stat outputs, then decode satoshi values into game fields.

Figure 4. Values and semantic fields are taken from the 2014 Coindroids transaction-response example. Source: [S1: Coindroids blog, “Introducing Coindroids \(2014\)”](#).

The early system also had a weakness that becomes a forensic caution. The 2014 post warned that Defcoin-QT change-address behavior could cause Coindroids to lose track of player identity unless Coin Control was used and change was forced back to the registration address. Source: [Coindroids blog, “Introducing Coindroids \(2014\)”](#). The 2017 creation guide still advised a separate wallet and explained that non-Coindroids transactions from the same wallet could put a wallet out of sync, requiring a resync of the “Ammo Clip.” Source: [Coindroids blog, “Creating a Droid”](#).

The practical lesson: do not over-cluster. A chain address, a Coindroids address role, and an inferred droid identity should be stored separately.

Figure 11. A false lead: ordinary wallet change can break droid identity

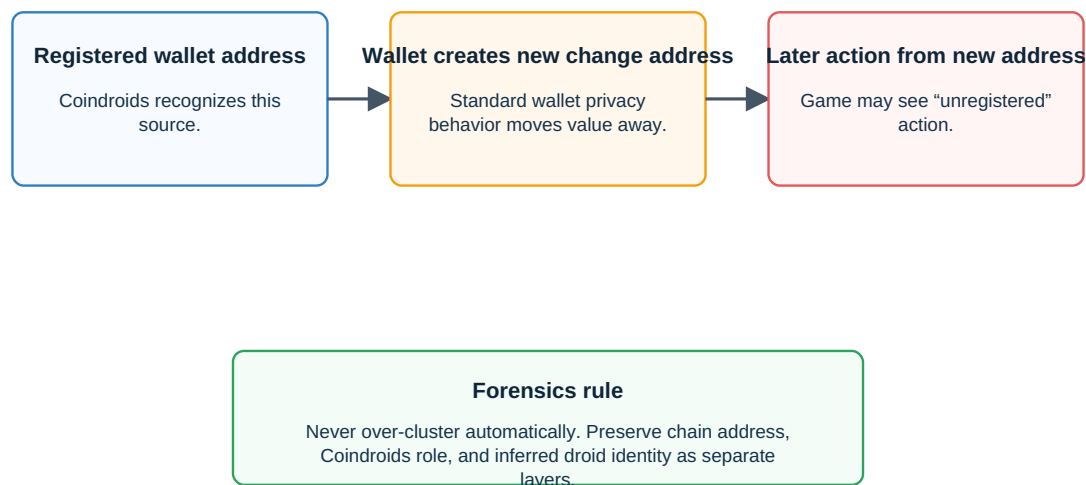


Figure 11. Wallet sync and out-of-sync behavior are described in the 2014 post and 2017 droid-creation guide. Sources: [S1: Coindroids blog, "Introducing Coindroids \(2014\)"](#) and [S6: Coindroids blog, "Creating a Droid"](#).

The Turn: 2017, When the Game Became Quieter

By 2017, Coindroids still used transactions as actions, but the forensic center of gravity had moved. The public documentation exposes fields such as `attack_address`, `primary_ammunition_clip`, and `additional_ammunition_clips` on the `/droid` endpoint. Source: [Coindroids Documentation](#). The attack guide says a user chooses an amount and sends it to the player's Attack Address. It also says the system processes the attack after confirmation and queues payouts until they reach a minimum threshold of **55,000 bits / 0.0055**. Source: [Coindroids blog, "Performing an Attack"](#).

That is the architectural pivot worth building around. The richer battle state moved toward the API and engine, while the chain carried standard value flows to purpose-built addresses plus settlement transactions.

The July 2017 mempool announcement adds another layer. Coindroids began ingesting Defcoin mempool data and exposing estimated pending actions, including registration/sync actions, item purchases, and attacks. But it explicitly says these were estimates: no payouts or inventory changes happened until the transactions appeared in a confirmed block. Source: [Coindroids blog, "Defcoin Mempool support added"](#).

Figure 3. The turn: from visible ledger speech to quieter settlement

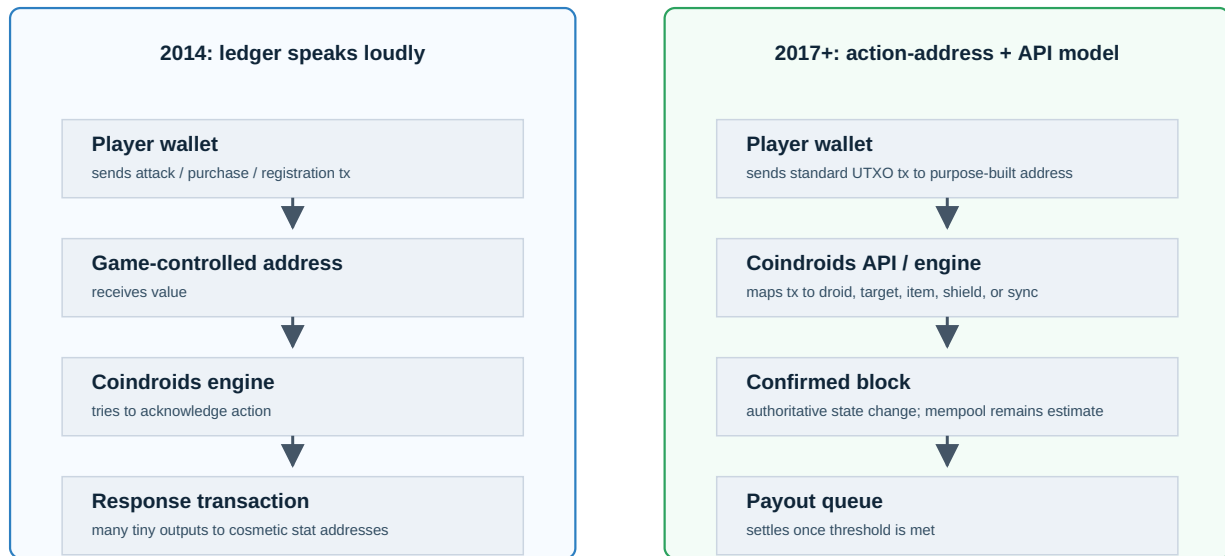
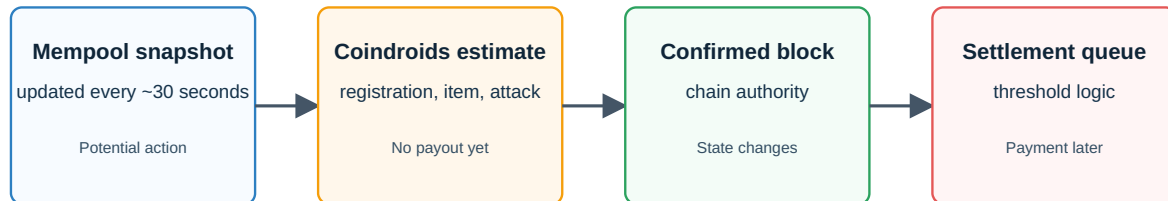


Figure 3. Left-side mechanics are described in the 2014 introduction, including response transactions and cosmetic addresses; right-side mechanics are supported by the 2017 activation guide, attack guide, mempool post, and API documentation. Sources: S1: Coindroids blog, “Introducing Coindroids (2014)”, S3: Coindroids blog, “Defcoin Mempool support added”, S4: Coindroids Documentation, S5: Coindroids blog, “Performing an Attack”, S6: Coindroids blog, “Creating a Droid”.

Figure 12. Pending is not proof: mempool estimates vs confirmed blocks



Design implication: the app may show pending Coindroids-like actions, but should only mark confirmed history after block inclusion.

Figure 12. The mempool post states pending actions are estimates, that the snapshot updates about every 30 seconds, and that payouts or inventory changes do not occur until confirmation. Source: [S3: Coindroids blog, “Defcoin Mempool support added”](#).

The Grammar of Coindroids Transactions

A forensic utility should classify Coindroids transactions as a set of address-role and outcome-role relationships, not as one monolithic “Coindroids transaction” type.

1. Registration and Ammo Clip Sync

Registration and sync transactions bind a wallet source to a droid. In 2014, that could include the 0.01 DFC one-time-address pattern. In 2017+, the activation guide describes wallet syncing as the final activation step and later describes resyncing the “Ammo Clip” when wallet identity breaks. Sources: [Coindroids blog, “Introducing Coindroids \(2014\)”](#) and [Coindroids blog, “Creating a Droid”](#).

2. Attack Outputs

A payment to a known droid attack address is the strongest attack fingerprint. The `/droid` endpoint documents `attack_address`; the attack guide describes sending the chosen amount to the player’s Attack Address. Sources: [Coindroids Documentation](#) and [Coindroids blog, “Performing an Attack”](#).

3. Item Purchase Outputs

The API and mempool examples include item purchase as an action type, and the DEF CON 25 recap reports 521 item purchases totaling 0.803 DFC. Sources: [Coindroids blog](#), “Defcoin Mempool support added” and [Coindroids blog](#), “Coindroids at DEF CON 25”.

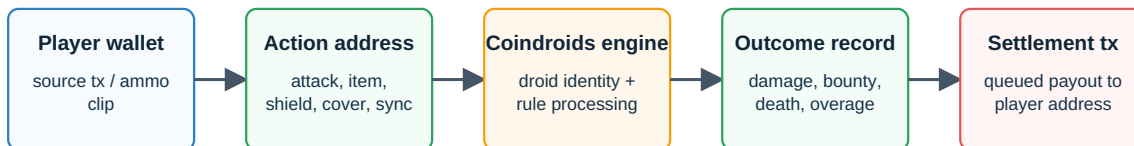
4. Shield and Cover Outputs

The currency endpoint documents `shield_cost`, `shield_address`, `cover_cost`, and `cover_address`. These are explicit destination-driven defensive-action fingerprints. Source: [Coindroids Documentation](#).

5. Payout Settlements

The payout endpoint exposes `action_block_height`, `settlement_txid`, `outcome_type`, `action_type`, `amount`, `address`, and `droid_id`. It includes examples such as “Droid destroyed” and “Clip size exceeded.” Source: [Coindroids Documentation](#).

Figure 5. The money trail: action, outcome, payout



Forensics view: a transaction is not just “sent DFC.” It may be an attack, a purchase, a defensive move, a sync event, or the settlement of earlier outcomes.

Figure 5. Flow derived from the attack guide, mempool post, and API event/payout fields. Sources: S3: [Coindroids blog](#), “Defcoin Mempool support added”, S4: [Coindroids Documentation](#), S5: [Coindroids blog](#), “Performing an Attack”.

Figure 9. Classification ladder: from raw Defcoin tx to Coindroids story

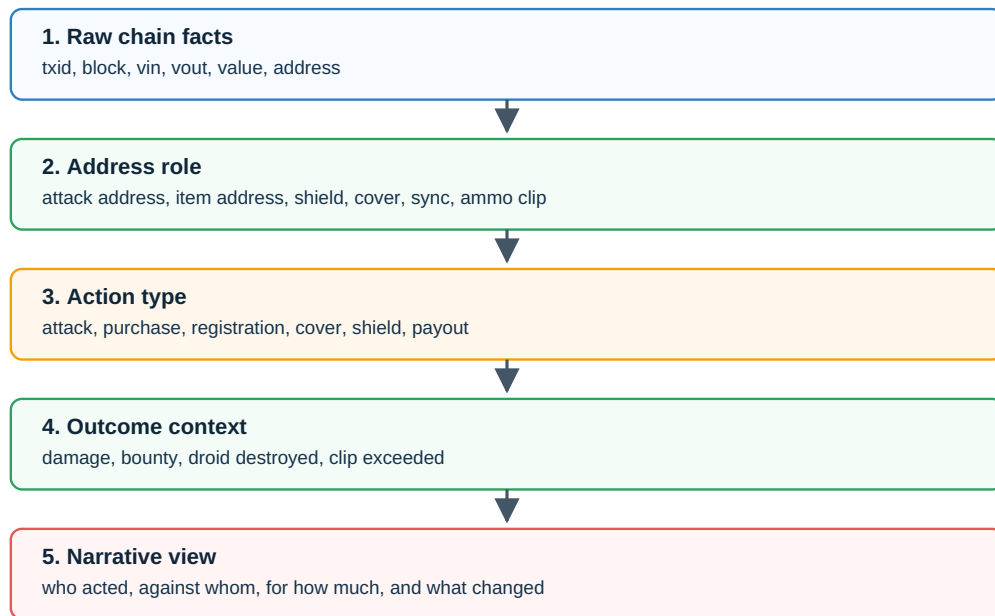
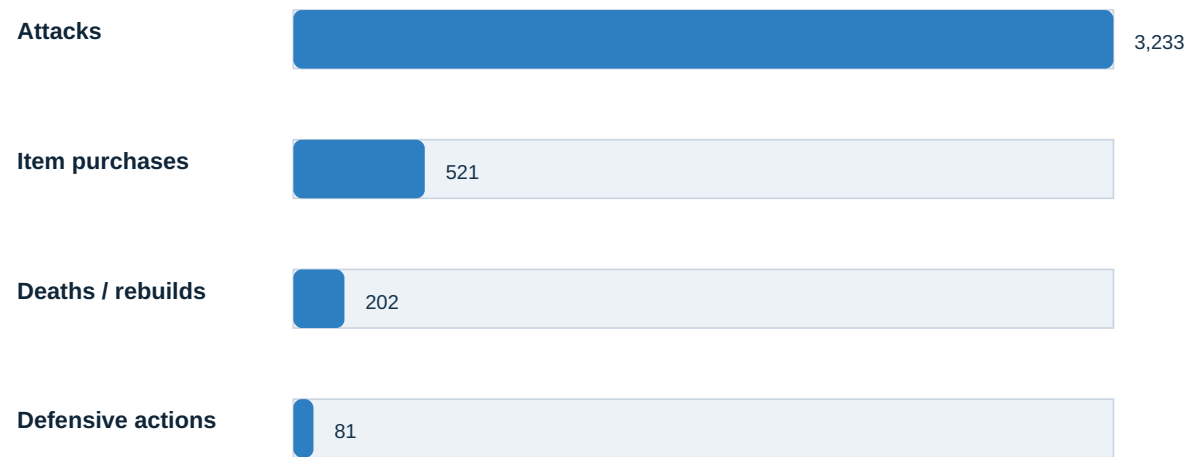


Figure 9. Classifier roles and action/outcome vocabulary are grounded in the 2014 post, 2017 guides, mempool post, and API documentation. Sources: S1: Coindroids blog, “Introducing Coindroids (2014)”, S3: Coindroids blog, “Defcoin Mempool support added”, S4: Coindroids Documentation, S5: Coindroids blog, “Performing an Attack”, S6: Coindroids blog, “Creating a Droid”.

What the Published Numbers Tell Us

DEF CON 25 is the strongest public benchmark because Coindroids published a compact statistical snapshot of Defcoin-realm activity. A scanner can use those numbers as a test suite: does the local classifier find roughly the same scale of activity in the event window?

Figure 6. What Coindroids did on Defcoin at DEF CON 25

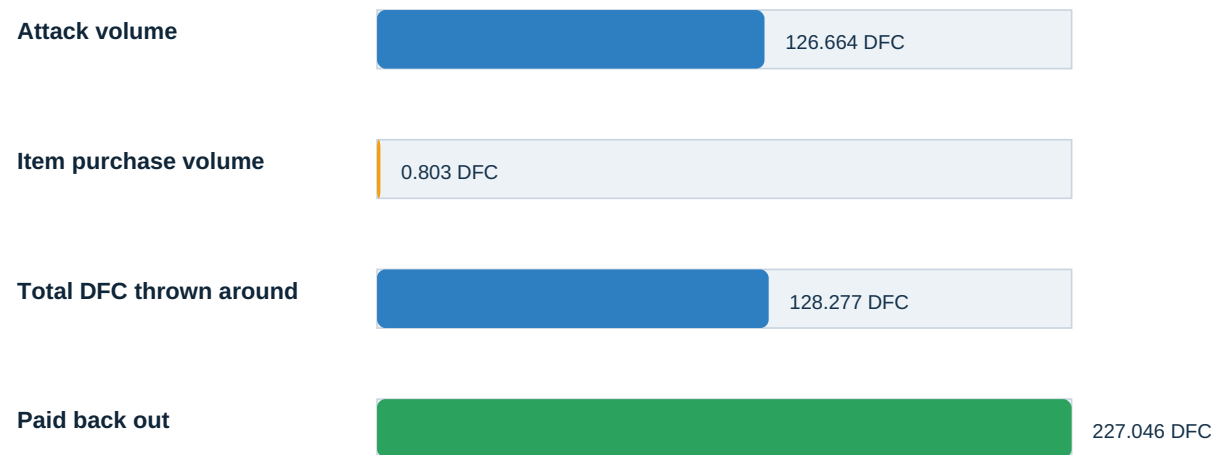


This figure shows published event counts, not inferred totals. A local classifier should try to reproduce these benchmarks from chain data.

Figure 6. All counts are from the DEF CON 25 Coindroids recap: 3,233 attacks, 521 item purchases, 202 deaths, and 81 defensive actions. Source: [S2: Coindroids blog](#), “Coindroids at DEF CON 25”.

The transaction count story and the value-flow story are different. Attacks dominate count and value among user actions. Purchases are numerous but low-value. Payouts can exceed event-period inputs because the system already held value available to win. That last point matters: a naive “money in vs. money out” ledger will misread the event unless it models bounties and pre-existing purses/bounties.

Figure 7. DEF CON 25 value moved through the game

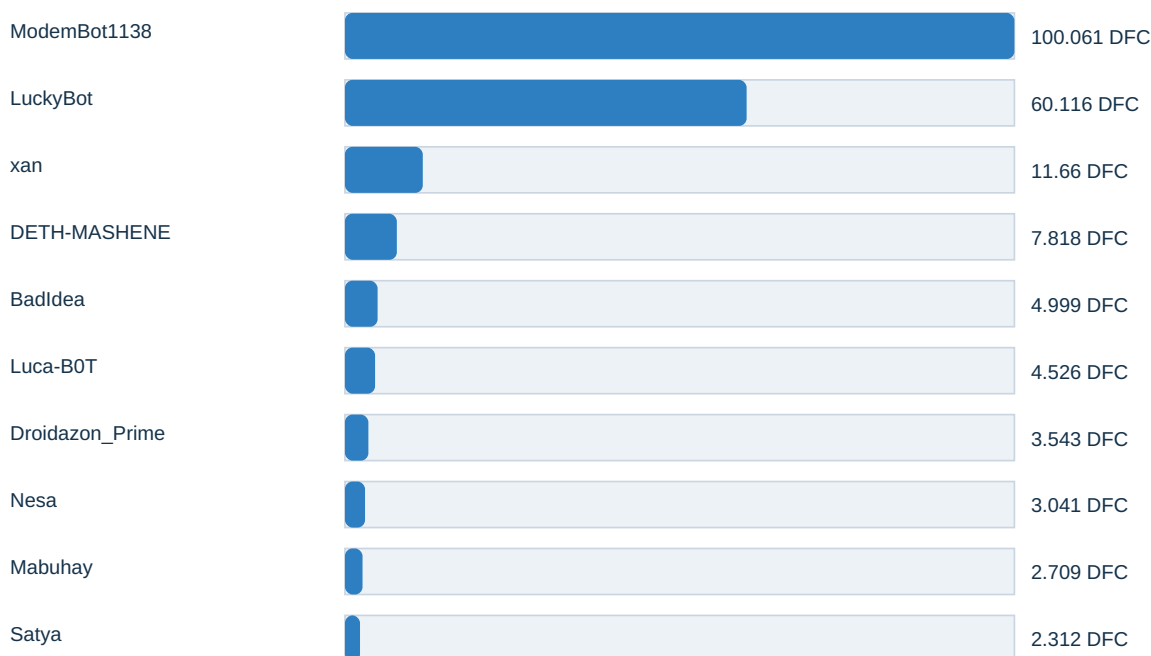


The payout bar exceeds the event input bars because the recap says "there was some to win in there already."

Figure 7. All DFC values are from the DEF CON 25 Coindroids recap, including the note that prior value existed in the game. Source: S2: Coindroids blog, "Coindroids at DEF CON 25".

The leaderboard is the most immediately human view. If a utility can rediscover ModemBot1138, LuckyBot, and the other published winners from chain classification, then it has moved beyond labels into reconstruction.

Figure 8. The published payout trail: DC25 top droids



Benchmark target: a chain scanner should be able to rediscover these published winners by classifying settlement payouts.

Figure 8. Top payout values are the published per-droid payout list from the DEF CON 25 recap. Source: [S2: Coindroids blog, "Coindroids at DEF CON 25"](#).

The Automation Problem

By DEF CON 27, automation was no longer merely suspected. Coindroids wrote that it introduced docker-droid, "a container with a bootstrapped defcoin client and built in coindroids bot," and added that this was the first time such automation was easily available to any player who wanted it. The same post says Coindroids activity represented 74% of the Defcoin network, down from about 99% in 2014. Source: [Coindroids blog, "Coindroids @ Defcon 27"](#).

The docker-droid README describes a simple bot that, by default, constantly attacked the weakest droid and notes that an empty order.txt defaulted to health_current.asc. Source: [GitHub, Abstract/docker-droid README](#).

A forensic utility should not claim to prove a bot from timing alone. It should assign an automation-candidate label based on converging signals: repeated block-cadence actions, weak-target selection, low manual variety, use of ignore filters, and long continuous activity.

Figure 13. Automation signals: useful, but never enough alone

Repeated block-cadence attacks	matches one-clip-per-block rhythm
Weakest-target selection	docker-droid README defaulted to health_current.asc
Low manual variety	few item or garage actions
Own-droid avoidance	ignore lists suggest multiple managed droids
Continuous overnight activity	behavioral anomaly, not proof by itself

The tool should label "automation candidate," not "bot proven," unless address ownership or code-derived evidence is available.

Figure 13. Automation at DEF CON 27 and the docker-droid bot are documented by Coindroids; docker-droid's README describes default weak-target behavior and attack filters. Sources: S8: Coindroids blog, "Coindroids @ Defcon 27" and S9: GitHub, Abstrct/docker-droid README.

The Utility App: What the User Should See

A user should not have to read raw JSON or infer meaning from addresses. The utility should let the user paste a txid, address, block range, or event window and answer a few plain questions:

- Was this transaction likely related to Coindroids?
- What action did it represent?
- Which droid or player was probably involved?
- Was the transaction part of an attack, purchase, shield, cover, sync, payout, or error recovery?
- What later payout or earlier action explains it?
- How confident is the classification, and why?

The tool should preserve a forensic chain of reasoning. Each classification should include evidence rows such as: "output pays known attack address," "amount within documented attack range," "source is known ammunition clip," "payout endpoint field names match settlement model," or "contains 2014 cosmetic response markers."

Figure 16. What a utility user should be able to discover

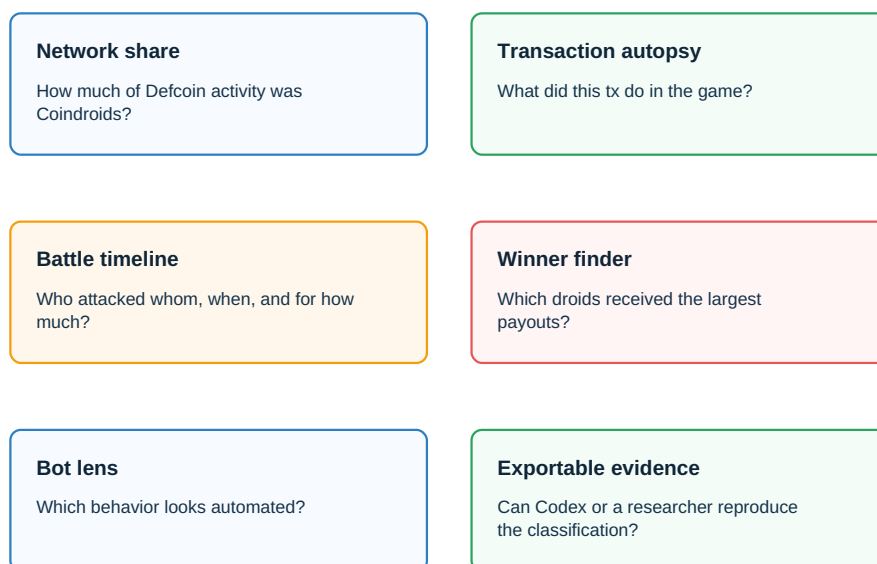


Figure 16. The proposed views are derived from the detection and reporting requirements established by the documented Coindroids mechanics and DEF CON 25 benchmarks. Sources: S1: Coindroids blog, “Introducing Coindroids (2014)”, S2: Coindroids blog, “Coindroids at DEF CON 25”, S3: Coindroids blog, “Defcoin Mempool support added”, S4: Coindroids Documentation, S5: Coindroids blog, “Performing an Attack”.

Implementation Specification

Minimum Tables

```
blocks(height, hash, time)
tx(txid, block_height, time, total_in_sats, total_out_sats, fee_sats)
vin(txid, n, prev_txid, prev_vout, address, value_sats)
vout(txid, n, address, value_sats, script_type)
coindroids_address(address, role, droid_id, droid_name, source, confidence)
coindroids_action(txid, vout, block_height, action_type, actor_id, target_id, amount_sats,
                  confidence, evidence_json)
coindroids_outcome(action_id, outcome_type, payout_txid, amount_sats, evidence_json)
```

Classifier Rules

High-confidence rules:

```
IF output address == known droid.attack_address
THEN classify output as Attack.

IF output address == known item purchase address
THEN classify output as Item Purchase.

IF output address == known shield_address
THEN classify output as Shield.

IF output address == known cover_address
THEN classify output as Cover.

IF tx contains multiple known 2014 cosmetic response markers
THEN classify tx as 2014 Response Telemetry.

IF payout destination == known ammunition clip or payout address
AND source cluster == Coindroids settlement wallet
THEN classify as Payout Settlement.
```

Medium-confidence rules:

```
IF amount == documented registration fee or observed sync amount
AND destination is one-time or sync-like address
AND source later performs valid Coindroids actions
THEN classify as Registration / Ammo Clip Sync.

IF attack amount exceeds known clip size
AND later payout returns related excess
THEN classify as Clip Exceeded / Refund Candidate.

IF same source repeatedly attacks at block cadence
AND target choice follows weak-health sorting
THEN mark Automation Candidate.
```

Figure 15. Minimal schema for a Coindroids forensic index

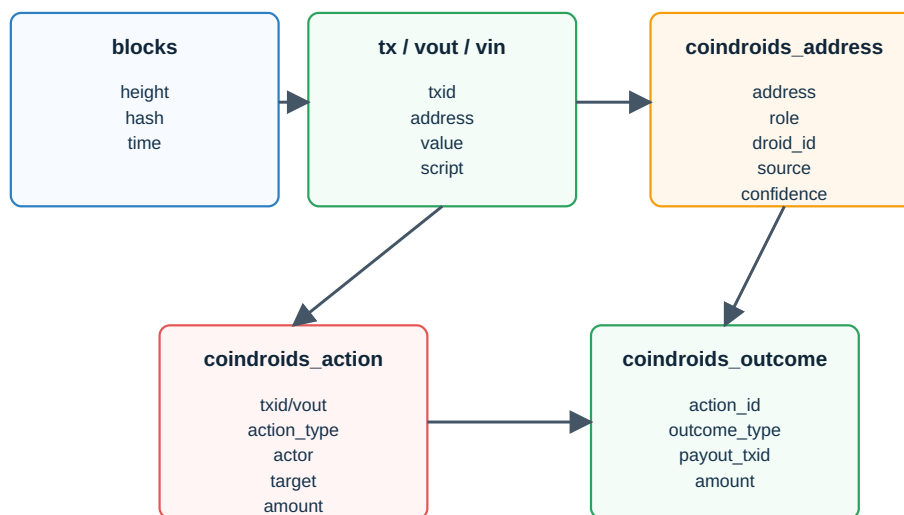


Figure 15. Table names are a proposed implementation, with fields based on documented chain facts, droid API fields, action types, outcome records, and payout fields. Source: [S4: Coindroids Documentation](#).

Future Work: Litecoin as a Harder Comparison Case

Coindroids was designed for UTXO currencies and documented both Defcoin and Litecoin realms. Source: [Coindroids blog](#), “[Supporting Other Currencies](#)”. That makes Litecoin a natural follow-up study, but it should come second. Defcoin is the better first target because the DEF CON reports provide public validation numbers and because event windows are easier to isolate. A Litecoin study would need stronger address seeds and stricter false-positive controls before claiming Coindroids-specific findings.

Figure 14. Why this study starts with Defcoin

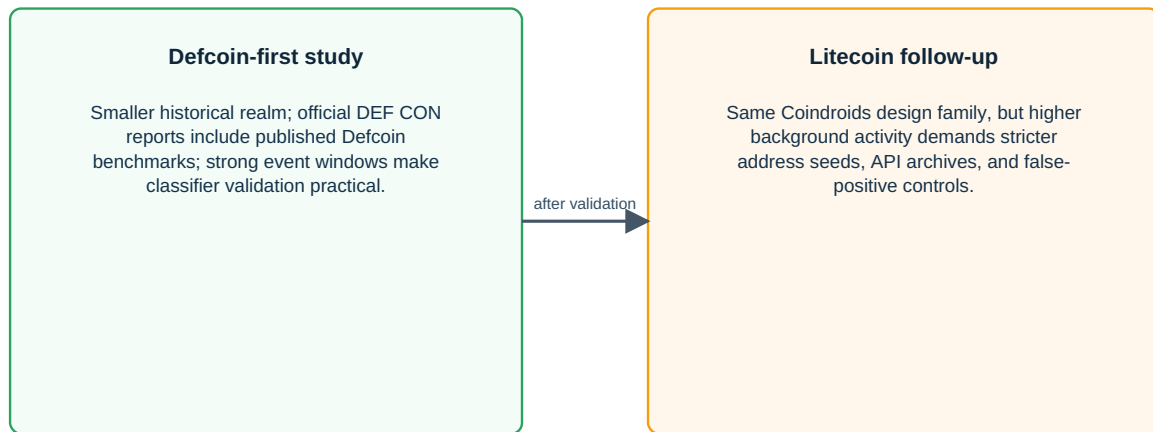


Figure 14. Coindroids documentation and blog material show Defcoin and Litecoin realms, while the DEF CON 25 report provides Defcoin-specific validation numbers. Sources: S2: [Coindroids blog, “Coindroids at DEF CON 25”](#), S4: [Coindroids Documentation](#), S6: [Coindroids blog, “Creating a Droid”](#), S10: [Coindroids blog, “Supporting Other Currencies”](#).

Conclusion: The Mystery Solved Enough to Build

The answer is yes: a Defcoin utility can reconstruct a meaningful Coindroids story from the chain.

The path is not a single magic signature. It is a layered reading:

1. Early Coindroids sometimes spoke through the ledger itself, using cosmetic response outputs.
2. Mature Coindroids used destination-driven UTXO commands, API state, confirmed-block processing, and payout settlements.
3. DEF CON event windows provide public benchmarks for validation.
4. Bot-era behavior adds timing and target-selection fingerprints, but those should be scored carefully rather than asserted recklessly.
5. The most valuable interface is not a raw explorer. It is a transaction autopsy and story reconstruction tool.

The point is not to romanticize the chain. The point is to make it legible. Coindroids left enough structure behind that Defcoin can be read not only as a ledger of payments, but as a record of play, strategy, automation, and payout economics.

Source Notes

- S1. [Coindroids blog, "Introducing Coindroids \(2014\)"](#). Supports: 2014 launch context, about 120 players, actions as transactions, 0.01 DFC registration-return pattern, transaction responses, cosmetic output markers, and wallet-change cautions.
- S2. [Coindroids blog, "Coindroids at DEF CON 25"](#). Supports: DEF CON 25 statistics, 71% network share, 65 active droids, 400k+ API requests, 3,233 attacks, 126.664 DFC attack volume, 521 purchases, 0.803 DFC purchase volume, 81 defensive actions, 202 deaths, 128.277 DFC thrown around, 227.046 DFC paid out, and the published payout leaderboard.
- S3. [Coindroids blog, "Defcoin Mempool support added"](#). Supports: Defcoin mempool ingestion, pending action estimates, update interval, and confirmation caveat.
- S4. [Coindroids Documentation](#). Supports: API fields including attack address, primary and additional ammunition clips, currency parameters, shield/cover addresses, action types, outcome examples, and payout fields.
- S5. [Coindroids blog, "Performing an Attack"](#). Supports: one clip per block, attack-address use, shield/cover/dead statuses, confirmed-block processing, and payout threshold.
- S6. [Coindroids blog, "Creating a Droid"](#). Supports: Defcoin/Litecoin realm choice, dedicated wallet guidance, activation through wallet sync, and unregistered/out-of-sync behavior.
- S7. [Coindroids blog, "Purse is now Bounty"](#). Supports: Purse-to-Bounty terminology and the concept that bounty increases as attacks are cast against a droid.
- S8. [Coindroids blog, "Coindroids @ Defcon 27"](#). Supports: DEF CON 27 automation, docker-droid introduction, Ballhair winner, all-time-high attack statement, and 2014-to-2019 Defcoin network share comparison.
- S9. [GitHub, Abstrct/docker-droid README](#). Supports: docker-droid bot description, default weak-target attack behavior, `health_current.asc`, and optional attack filters.
- S10. [Coindroids blog, "Supporting Other Currencies"](#). Supports: Coindroids as a UTXO-compatible design, Defcoin as the yearly DEF CON realm, and Litecoin support.
- S11. Robert McKee, *Story: Substance, Structure, Style, and the Principles of Screenwriting*. HarperCollins, 1997. Used as a writing-structure influence only: inciting incident, imbalance, progressive complications, turning point, and resolution.
- S12. Eugene L. Lowry, *The Homiletical Plot: The Sermon as Narrative Art Form*. Westminster John Knox, revised edition 2001. Used as a writing-structure influence only: delayed disclosure, movement from discrepancy toward clue and consequence, and the report's "question before answer" structure.

Table of Contents

1. Money Was the Move
2. The Forensic Question
3. The Trail Begins: 2014, When the Chain Spoke Back
4. The Turn: 2017, When the Game Became Quieter
5. The Grammar of Coindroids Transactions
6. What the Published Numbers Tell Us
7. The Automation Problem
8. The Utility App: What the User Should See
9. Implementation Specification
10. Future Work: Litecoin as a Harder Comparison Case
11. Conclusion
12. Source Notes
13. Table of Contents